

System Description

****Status:**** Production readiness review

****Last Updated:**** 2026-06-01

Executive Summary

This document describes the system architecture, primary components, data model, security and operational controls, deployment and recovery posture, and key operational runbooks for the platform. The system is composed of a single-page frontend, a trusted server-side backend, and managed deployment infrastructure.

Goals and Non-Goals

- Goals:
 - Provide reliable, auditable business operations with server-side enforcement of rules.
 - Support real-time subscriptions and transactional operations.
 - Support packaging the web frontend into platform wrappers (mobile/desktop) while keeping server authority centralized.
- Non-Goals:
 - Offload business logic to clients.
 - Integrate third-party payment processors or external accounting systems in the core platform (handled via separate adapters if required).

High-Level Architecture

- Frontend: Next.js (App Router). Responsible for UI, routing, PWA support, and platform packaging via Capacitor/Tauri. Treated as untrusted.
- Backend: Convex. Single source of truth for domain data, enforces business logic, authorization, transactions, and immutable audit events. Treated as trusted.
- Infrastructure: Vercel for frontend (hosting & env management), Convex for backend hosting and database. Operators also manage packaging pipelines for Android and Windows.

```
```mermaid
flowchart LR
 A[User Browser / App] -->|HTTPS| B[Next.js Frontend]
 B -->|Convex Requests| C[Convex Backend]
 C --> D[(Convex DB)]
 C -->|Real-time| B
 B --> E[Capacitor / Tauri (optional packaging)]
 subgraph Infra
 V[Vercel]
 end
```

```
 ConvexSvc[Convex]
 end
 B -. hosted on .-> Vercel
 C -. hosted on .-> ConvexSvc
 ...
```

## ## Component Responsibilities

- ``Frontend (Next.js)``:
  - Render UI by role, manage local view state, and provide offline-capable UX (PWA caching strategy).
  - Collect inputs and send to backend; never enforce critical business rules.
  - May implement local offline queue for UX, but authoritative replay/validation happens server-side.
- ``Backend (Convex)``:
  - Authoritative data store and business logic executor.
  - Enforces authorization and validation for all mutations and queries.
  - Generates UTIDs for auditability, stores immutable logs for admin actions, ledger entries, and rate limit hits.
  - Handles atomic transactions and complex multi-entity operations.
- ``Deployment Infrastructure``:
  - Vercel: Hosts the frontend builds and serves the PWA.
  - Convex: Hosts backend functions and stores data.
  - Packaging pipelines: Build web artifacts and produce platform-specific wrappers (Capacitor for Android, Tauri for Windows).

## ## Data Model Overview

Primary entities (stored in Convex collections):

- Users
- Listings / Products
- Inventory / TraderInventory
- Ledger entries / WalletLedger (immutable)
- Purchases / BuyerPurchase
- ListingUnit (individual saleable units)
- AdminAction (immutable audit logs)
- SystemSettings
- Notifications
- RateLimitHit (immutable)

Immutability conventions:

- Financial or audit records are immutable (append-only) to ensure traceability.

- Operational entities may change state, but transitions should be logged with UTIDs.

## ## Authentication & Authorization

- Authentication: Stateful sessions managed by Convex Auth. Production-grade authentication must be enabled before go-live; current pilot usage may use weaker controls.
- Authorization: Role-based; all checks performed on server. Admin-level functions are gated and recorded in ``AdminAction`` with UTIDs.
- Role assignment: Must be explicit (no email-prefix inference). Admins change roles via authorized server-side mutations.

## ## Business Rules & Transaction Model

- All money-moving and inventory-moving mutations are blocked when ``systemSettings.pilotMode === true`` (kill-switch).
- Purchase window state (``purchaseWindows.isOpen``) controls buyer purchases.
- Mutations that perform multi-entity updates must run inside Convex transactions to ensure atomicity.
- UTIDs are generated for any meaningful action to form a complete audit trail.

## ## Offline and Sync Behavior

- Clients may maintain a local queue (IndexedDB or localStorage) for offline operations.
- On reconnect, the client replays the queue to Convex; the backend validates idempotency and enforces server-side conflict policy (recommendation: server-wins plus discrepancy reporting).
- Sync states should include: pending, synced, failed, conflict.

## ## Observability and Monitoring

- Telemetry:
  - Backend function logging for key mutation paths and errors.
  - Metrics for mutation latency, error rate, queue backlog, and active real-time subscriptions.
- Alerts:
  - High error rate on money-moving mutations.
  - Backup failures or restore errors.
  - Convex service unavailability and subscription disconnects.

## ## Backup, Restore, and Data Recovery

- Current status: Convex-managed backups are assumed but operator-level backup/restore procedures must be verified and documented.
- Required actions before production:
  - Verify Convex backup frequency and retention; obtain operator-run restore runbook.
  - Implement periodic backup verification tests.

## ## Security Controls

- Secrets & Environment:
  - Store secrets in platform environment variables (Vercel, Convex env). Limit access to operators.
- Data Protection:
  - Sensitive data must not be cached on clients.
  - Use transport-level encryption (HTTPS/TLS) for all client-server communications.
- Access Control:
  - Least privilege for operator accounts and CI tokens.
  - Admin actions recorded immutably.

## ## Operational Playbooks

- Deployments:
  - Frontend: Build and deploy via Vercel pipeline; tag releases.
  - Backend: Deploy Convex functions using the repository's Convex deploy process.
- Incident Response:
  - If Convex is unavailable: declare outage, disable money-moving UI flows, and notify operators.
  - If data corruption suspected: contact Convex support and follow restore runbook.
- Kill-Switches:
  - ``pilotMode`` (admin): blocks money/inventory mutations; recorded in ``adminActions``.
  - ``purchaseWindow`` (admin): toggles buyer purchase availability.

## ## External Dependencies and Risk Ownership

- Vercel: frontend hosting – availability risk borne by system operator.
- Convex: backend hosting and database – availability and data integrity risk borne by system operator.
- Any external payment or third-party integrations should be isolated and treated as optional adapters.

## ## Known BLOCKED Items & Gaps

- Production authentication activation and password/email delivery must be implemented and verified.
- Backup & restore operator runbook for Convex is unknown and must be validated.
- Delivery verification and storage-fee automation interfaces are not fully implemented/verified.
- System operator action logging (infrastructure-level events) is currently UNKNOWN and should be addressed.

## ## Compliance & Audit

- Maintain immutable `AdminAction` and `WalletLedger` logs for auditors.
- Provide exportable audit trails referencing UTIDs for all meaningful money and inventory movements.

## ## Final Checklist Before Production

- [ ] Verify production authentication and role assignment mechanics.
- [ ] Confirm Convex backup & restore runbook and test restores.
- [ ] Implement/verify pilot mode enforcement in backend.
- [ ] Implement delivery verification with audit logging.
- [ ] Establish monitoring and alerting for critical mutation error rates.

---

## ## Appendix

- Primary files and locations:
  - `architecture.md` (system architecture notes)
  - `DOMAIN\_MODEL.md` (entity definitions)
  - `BUSINESS\_LOGIC.md` (rules and irreversible actions)